
Artificial Intelligence

Tut/Lab week 3

COSC2129

Outline

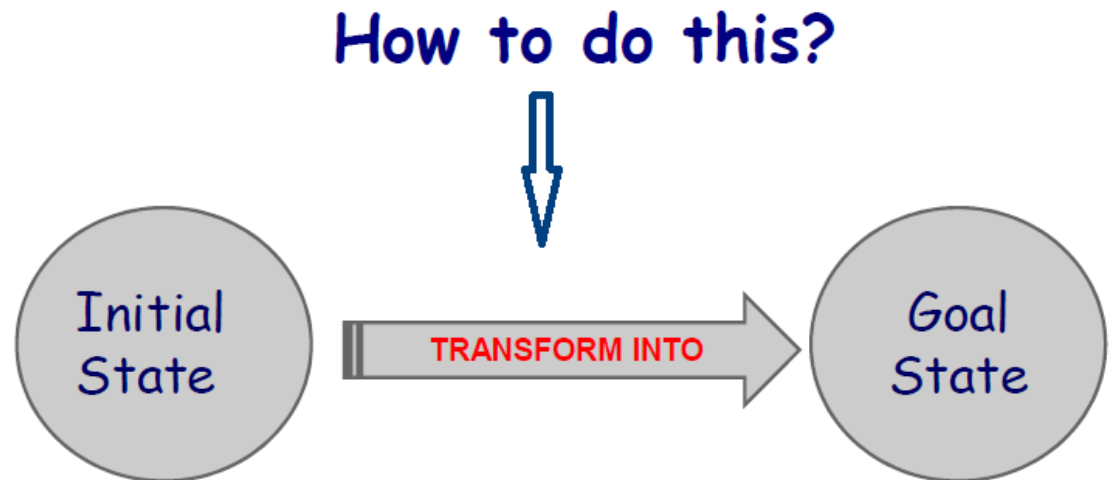
- Highlights:
 - Problem solving by search
 - Search & games
 - Adversarial search
- Hands-on exercise
- Assignment 1

Material

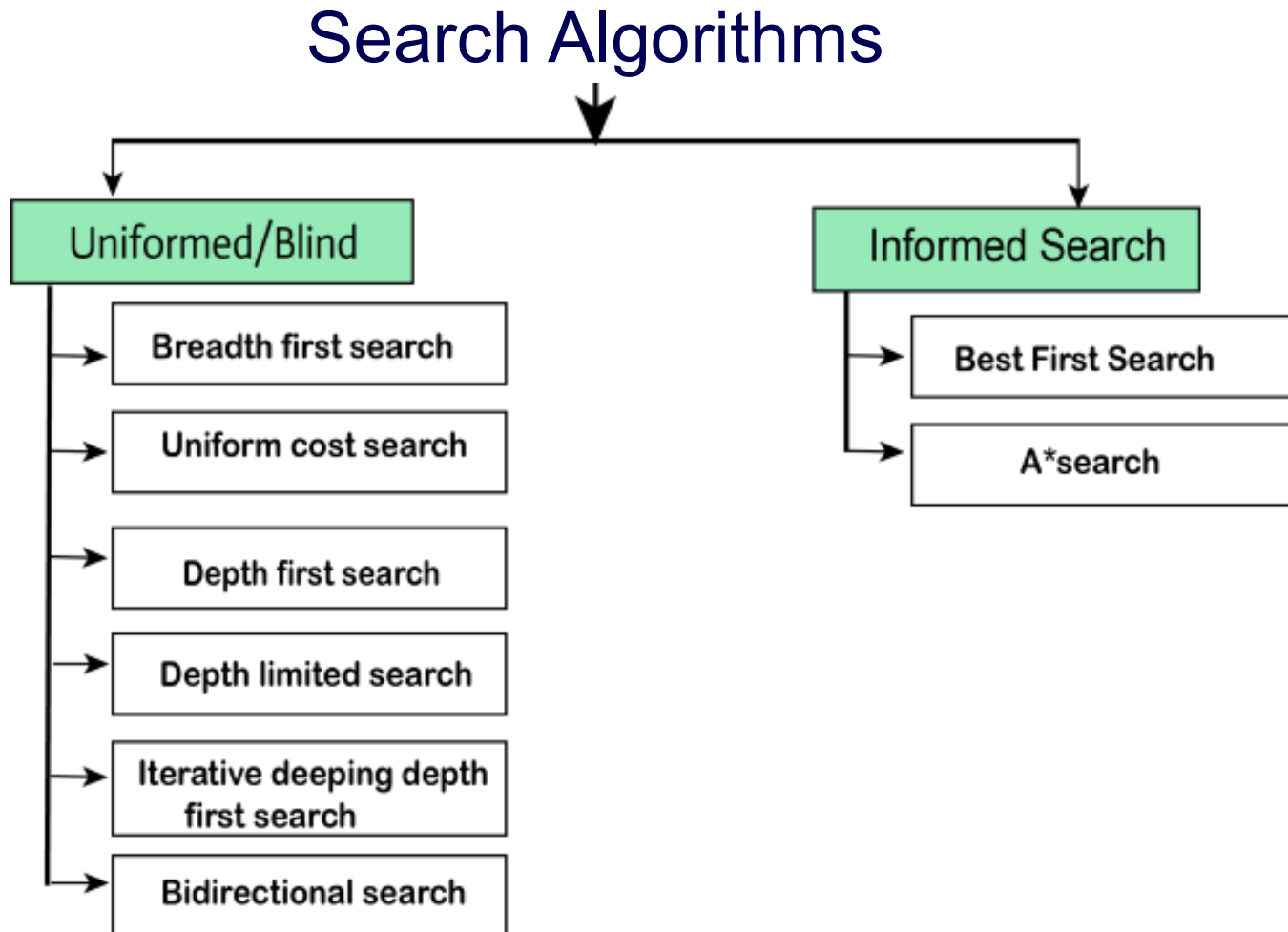
- Chapter 6, Book: Artificial Intelligence: A Modern Approach Global edition
- Book - TOC of Artificial Intelligence: A Modern Approach, 4th Global ed.
- AIMA: Pseudo code
- Good practices Python

Search problems

- A search problem is defined by:
 - Possible states (search space)
 - Initial state
 - Actions
 - Transition mode
 - Goal test
 - Path cost



Blind search algorithms



Performance of search strategies

- **Completeness:** Is it guaranteed to find a solution if one exists?
- **Optimality:** Will it find an optimal solution?
- **Time complexity:** How many nodes get expanded?
- **Space complexity:** How much memory is needed?
- **Time and space complexity is calculated based on:**
 - b : Branching factor
 - d : Depth of the shallowest goal node
 - m : Maximum length of any path in the state space.

Games and search

Games

- Initial State s_0
- $\text{Player}(s)$: who moves in state s
- $\text{Action}(s)$: Legal moves in s
- $\text{Result}(s,a)$: transition model
- $\text{Terminal-test}(s)$: true if the game is over
- $\text{Utility}(s)$: utility for s if s is terminal



Minimax search

Two players: MAX and MIN

Initial state is a MAX node

Children of MAX state are MIN states

Children of MIN state are MAX states

Calculate Minimax(s)

- Utility(s) if Terminal-test(s)
- $\text{Max}_a \text{Minimax}(\text{Result}(s,a))$ if $\text{Player}(s) = \text{MAX}$
- $\text{Min}_a \text{Minimax}(\text{Result}(s,a))$ if $\text{Player}(s) = \text{MIN}$
- Calculate in depth-first manner

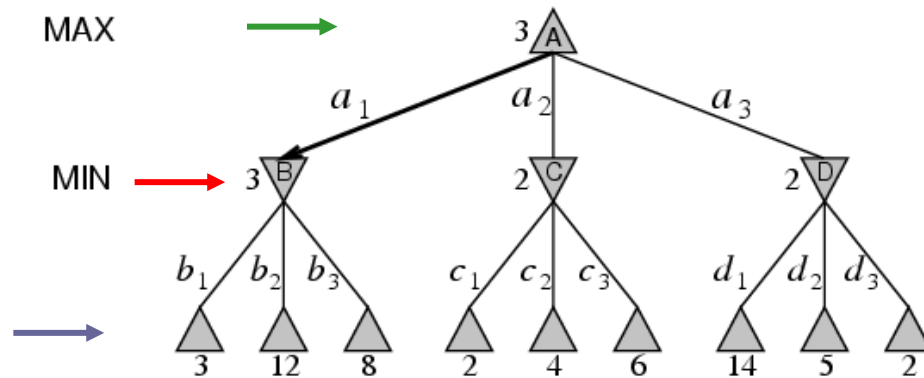
Minimax search

- Leaf node: $\text{Minimax}(s) = \text{Utility}(s)$
- MAX node: $\text{Minimax}(s)$ is maximum of children
- MIN node: $\text{Minimax}(s)$ is minimum of children

“I choose best move for me, assuming my opponent chooses the best move for him/her”

Minimax search example

Assume, for example, a game that only has two moves from start to finish:



- ▲ is a "MAX node". It is MAX's turn to move (to find the max value).
- ▼ is a "MIN node". It is MIN's turn to move (to find the min value).

Alpha-beta pruning

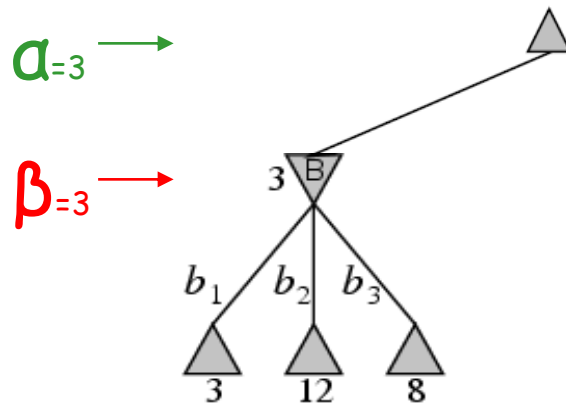
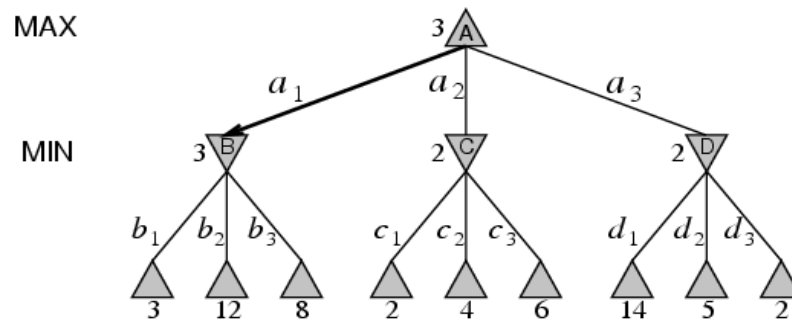
Same search as for minimax except:

MIN node: stop if current value $\leq$ alpha of parent
(this node can't be higher than alpha)

MAX node: stop if current value $\geq$ beta of parent
(this node can't be lower than beta)

This means that the whole subtree for a pruned node is not expanded

Alpha-beta pruning



- Expand the first branch.
- Get evaluation values for each leaf node.
- So the value for B (β) is 3.
- So the root value (α) is currently 3.

Games and search

- Set up game as a kind of search tree
- Use Minimax method to choose moves
- Use alpha-beta pruning to cut down alternatives
- Some implementation tricks such as transposition tables as hash tables
- So ...

Games and search

- Games involve search, but ...
- Search space can be huge (chess 10^{40} states (!))
- Not all moves may be chosen by opponent
- Time limits may limit search
- Exhaustive search is not how humans do it!

Games and search

- Assume optimal opponent
- Prune search space
- Use heuristic approximations
- Deal with imperfect information and chance

Evaluation Function

Limit depth and estimate utility via a
heuristic evaluation function (educated guess)

Calculate $H\text{-Minimax}(s)$

- $\text{Eval}(s)$ if $\text{Cutoff-test}(s,d)$
- $\text{Max}_a H\text{-Minimax}(\text{Result}(s,a),d+1)$ if $\text{Player}(s) = \text{MAX}$
- $\text{Min}_a H\text{-Minimax}(\text{Result}(s,a),d+1)$ if $\text{Player}(s) = \text{MIN}$

So what is a good evaluation function?

- Efficient
- Match utility on terminal states
- Correlate with winning chances on non-terminal states
- Will have some level of uncertainty (as it is an estimate)

Evaluation Function

Often a **weighted sum** of factors

$$\text{Eval}(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s)$$

Assumes factors are **independent**

Common chess function:

$$9 * \text{\#Queens} + 4 * \text{\#Bishops} + 3 * \text{\#Knights} + \text{\#pawns}$$

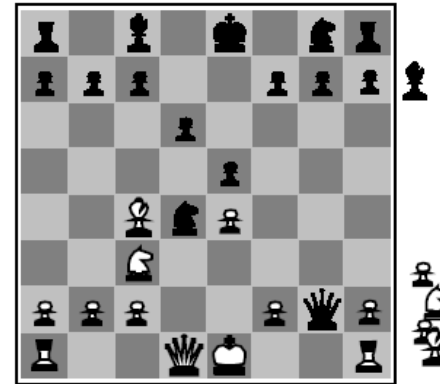
Says nothing about the position of chess pieces.

Evaluation Function



Black to move

White slightly better



White to move

Black winning

For chess the weights and factors could be:

$$w_1 f_1(s) = 9 * (\text{number of white Queen} - \text{number of black queen})$$

$$w_2 f_2(s) = 4 * (\text{number of white Bishop} - \text{number of black Bishop})$$

$$w_3 f_3(s) = 3 * (\text{number of white knight} - \text{number of black knight})$$

$$w_4 f_4(s) = 1 * (\text{number of white pawn} - \text{number of black pawn})$$

... so on and so forth...

By this simple function we can estimate which player is closer to winning.

Activity 2: Hand-on Practice

- Tut3 Hand-on exercise

Assignment 1

Question?